BWC reporting: step-by-step implementation guide

Printable companion: download the PDF. Use this Markdown guide when copying expressions into Power Automate.

Updated October 14, 2026. Follow these steps in order using your organization's Microsoft 365, SharePoint, and Power BI tenant, including its government-cloud portals where applicable.

The completed process runs **Power Automate → Axon API → Excel Online → SharePoint → Power BI Service**. Power BI Desktop is used for initial report authoring. The scheduled process runs in the cloud after publication.

The repository supplies scripts, a workbook, API contracts, and an editable report project. You must create the tenant connections and Power Automate flow. The expression JSON is a reference, not an importable flow. Live API access, tenant execution, and Desktop loading/rendering still require verification.

Implementation sequence

Stage	Steps	Result
Prepare	1–4	Current clone, sample report, SharePoint folders, and scripts.
Test processing	5	A synthetic cloud run with expected counts.
Build the flow	6–14	Backfill and incremental runs publish completed workbooks.
Connect Power BI	15–17	The report reads SharePoint and refreshes in the Service.
Operate	18–19	Recurrence, full reconciliation, and recovery.

MVP rules: **normal 0–14, moderate 15–29, critical 30+ calendar days since upload**. Recording creation starts **March 11, 2024**. Active and Queued for Deletion evidence is eligible; assigned categories exclude an item. All evidence types remain in scope. Calendar upload age does not measure the separate three-business-day policy.

Step 1 — Gather access and connection details

Keep these values in approved private configuration. Agency host/IDs already supplied for this project are not published in the repository.

Item	Required value or access
Axon base URL	Your agency's HTTPS origin, without `/api`, a query, or embedded credentials.
Agency/partner ID	The agency ID supplied for the project.
Client ID and secret	The reporting API client's credentials; use Step 6 for the secret.
Axon permission	**Reports → read**. The Interview permission in one attachment is a different permission.
SharePoint	Site home-page URL, library display name/URL path, and reporting folder.
Power BI	Destination workspace and publishing/refresh access.

Item	Required value or access
Schedule	Daily or weekly; this guide starts with 6:00 AM Eastern.

The flow identity needs Edit access to the reporting folder, access to the Excel Online scripts, and authorization to request the Power BI model's refresh. The Power BI source identity needs Read access to SharePoint. Select ownership/connections that can continue operating when the original author is unavailable, according to your organization's account policy.

Confirm that the tenant permits **Excel Online (Business) → Run script**, **HTTP** or an approved API connector, and Power BI publishing/refresh. HTTP/custom connectors commonly require Power Automate Premium. Resolve unavailable actions or licensing before building the live flow.

Checkpoint: private bindings, destination, workspace, identities, and connector availability are recorded. Credentials are not required to inspect the synthetic report.

Step 2 — Get the current repository and inspect the report

For a new checkout:

```
git clone https://github.com/CheesesteakPapi/RScriptDashboard.git
cd RScriptDashboard
```

For an existing checkout, close Power BI and inspect your local changes before updating:

```
git status
git pull
```

Preserve your own edits if Git reports a conflict. Extracting a fresh ZIP into a new folder is an alternative to updating an edited checkout.

1. Open powerbi/template/BWC Uncategorized.pbip in a current Power BI Desktop release.
2. Keep its `BWC Uncategorized.Report` and `BWC Uncategorized.SemanticModel` folders beside the PBIP file.
3. Select **Refresh**. `UseSampleData` starts as `true`; the sample is embedded.
4. Inspect **Overview**, **Remediation**, and **Data Quality**.
5. Verify **8 total, 4 critical, 2 moderate, 2 normal, and 3 rejected records**, with **SYNTHETIC DEMO DATA** displayed.
6. Use **File → Save As → Power BI file (.pbix)** for a compiled copy if needed. Keep the project source for future editing.

The sample snapshot stays fixed at **October 13, 2026**, even when opened later.

Checkpoint: the sample opens and refreshes in Desktop. For the earlier `version.json` error, update the checkout: the corrected project contains `BWC Uncategorized.Report/definition/version.json`. Renaming a ZIP or PBIP to PBIX does not compile a report.

Step 3 — Prepare SharePoint

1. Open SharePoint from the organization's Microsoft 365 portal.
2. Open the chosen team site and its **Documents** library.
3. Select **New → Folder**, create **BWC Reporting**, and open it.
4. Create **templates**, **raw**, and **latest** subfolders.
5. Open **templates → Upload → Files** and upload only automation/report-template.xlsx.
6. Use **Manage access** for the intended reporting team and give the flow identity Edit access.

Initial layout:

```
Documents / BWC Reporting /
    templates /
        report-template.xlsx
    raw /
    latest /
```

The flow creates:

```
raw / YYYY-MM-DD / RUN-ID /
    evidence-api.json
    Uncategorized Report.xlsx
latest /
    Uncategorized Report.xlsx
```

Upload the single workbook file. Install the `.office.ts` files as scripts in Step 4. Use the PBIP folders for report authoring.

For SharePoint file-path fields, include the library and reporting folder, for example `/Shared Documents/BWC Reporting/templates/report-template.xlsx`. In Excel Online actions, select the site and library separately and supply the file Identifier/Id.

Record the site home-page URL, for example `https://YOUR-TENANT.sharepoint.com/sites/YOUR-SITE`. The displayed Documents library commonly has URL path `Shared Documents`; verify yours. A sharing link to an individual file is not the site URL parameter.

Checkpoint: `templates/report-template.xlsx` opens in Excel for the web; raw/latest folders are ready for runs.

Step 4 — Install both Office Scripts

1. Open the uploaded template in **Excel for the web**.
2. Select **Automate → New Script**.
3. Replace the starter code with the full contents of automation/ProcessEvidence.office.ts.
4. Save it as **ProcessEvidence**.
5. Create another script with automation/ReadRefreshState.office.ts.
6. Save it as **ReadRefreshState**.
7. Verify the flow's Excel Online connection identity can select both scripts. Workbook sharing and script ownership/sharing are separate; use the tenant's supported script-sharing procedure.

The scripts process data; they have no client-secret parameter. Axon authentication belongs to the acquisition flow.

Checkpoint: both scripts save without editor errors and appear in the Excel Online Run script action.

Step 5 — Test the processor in the cloud with synthetic data

This isolated test verifies SharePoint and Excel Online before Axon. It does not update latest.

1. Create `BWC Reporting/raw/smoke-test` in SharePoint.
2. In Power Automate, choose **Create → Instant cloud flow**, name it **BWC - Synthetic smoke test**, and use **Manually trigger a flow**.
3. Add **SharePoint → Get file content using path** for the template.
4. Add **SharePoint → Create file** in raw/smoke-test, named `Uncategorized Report.xlsx`, using the preceding **File Content**.
5. Add **Excel Online (Business) → Run script** for that new file. Use its **Identifier/Id**, not the numeric SharePoint ItemId.
6. Select ProcessEvidence and enter:

Parameter	Test value
csvText	Entire contents of examples/axon-pages.sample.json, including outer braces.
snapshotDate	`2026-10-13`
runId	`smoke_20261013`
sourceDataDate	Empty string; use `''` in Expression when needed.
sampleData	Boolean `true`.
sourceFormat	`axon-page-bundle`

Save and test. Inspect the action's result and open the working workbook. For another test, use a new folder/filename to avoid a Create file collision.

Checkpoint: 8 uncategorized, 2 normal, 2 moderate, 4 critical, 3 rejected. Those three rejections are intentional fixture rows, not a production quality target.

Optional incremental test: process axon-backfill.sample.json with sourceFormat=`axon-refresh-bundle`, snapshotDate=`2026-10-13`, runId=`initial`, and sampleData=true. Copy that processed workbook to another test location. Process axon-incremental.sample.json on the copy with snapshotDate=`2026-10-14`, runId=`second`, and the same format/sample flag. The second run has **3 items, 1 per age band**, and **5 EvidenceState records**. Keep both test workbooks outside latest.

Step 6 — Configure Axon authentication

Create a separate **Instant cloud flow** named **BWC - Axon connection test**, using Manually trigger a flow. Test authentication here, then reuse the protected configuration in the production flow in Step 7. Keep production in pilot use until the live checks pass.

Production credential source

Use the organization's approved connection or secret store to retrieve the client secret at runtime. This repository does not create that credential source; its connector/action depends on your tenant. Ask the administrator for the specific approved credential action if none exists. No Azure compute host is needed for this process.

Protect credential retrieval, token responses, and requests carrying Authorization with the supported **Secure Inputs/Outputs** settings. If a variable action cannot hide sensitive values, reference the secure credential action output directly instead. Confirm masking with dummy values before entering live credentials.

Temporary MVP authentication test

For a controlled test, a secret can be entered in the token HTTP action's expression. Restrict flow editors: a literal secret remains visible in the flow definition. Secure Inputs/Outputs hides run-history values; it does not create a secret vault. Replace the literal with an approved runtime source for production, and keep it out of exported definitions/source control.

1. Add **HTTP**, rename it **Get_Axon_Token**, and enable Secure Inputs/Outputs in Settings.
2. Set these fields:

Field	Value
Method	`POST`
URI	Your HTTPS Axon base URL followed by `/api/oauth2/token`.
Authentication	None for this form-body client-credentials request.

Field	Value
Headers	Content-Type = `application/x-www-form-urlencoded`

3. In **Body → Expression**, replace placeholders only inside the protected test flow:

```
concat(
  'grant_type=client_credentials&client_id=',
  uriComponent('YOUR_CLIENT_ID'),
  '&client_secret=',
  uriComponent('YOUR_CLIENT_SECRET')
)
```

For production, use `token_body` in automation/axon-flow-expressions.json after binding ClientSecret to your approved runtime source. Every value is URL-encoded. In the designer's **Expression** tab omit the leading `@` included in JSON/code-view fragments.

The older vendor guide includes `partner_id`; the shorter authentication document omits it. Start with LegacyTokenPartnerId=false and confirm against the agency's current client documentation. If that tenant requires the older form, set it true; the prepared expression appends the encoded AgencyId. Diagnose errors rather than repeatedly toggling the setting.

4. Validate a successful response with nonempty access_token and positive expires_in. A token-request 401 fails authentication.
5. Bind AccessToken to `body('Get_Axon_Token')?['access_token']` after validating the response. Track token expiry from:

```
formatDateTime(
  addSeconds(utcNow(), int(body('Get_Axon_Token')?['expires_in'])),
  'yyyy-MM-ddTHH:mm:ssZ'
)
```

6. Reuse the token across pages. After any successful renewal, replace AccessToken and expiry with the new response's values. Renew shortly before expiry and once after a report-request 401. Do not assume a fixed lifetime.

Checkpoint: token acquisition succeeds and sensitive values are hidden in run history. Reports.read is tested by the first report GET.

The earlier Codex `AXON_CLIENT_SECRET` environment requirement is for development/testing. Entering it in Codex does not configure Power Automate, Power BI, or Excel.

Step 7 — Create the production flow and variables

1. Select **Create → Scheduled cloud flow**, name it **BWC - Refresh Orchestrator**.
2. Select daily or weekly recurrence at **6:00 AM**, timezone **Eastern Standard Time**. This zone includes Eastern daylight-saving transitions.
3. Enable trigger **Concurrency Control**, degree **1**, and use this as the sole writer of latest.
4. Keep recurrence disabled outside controlled pilot tests. Use its supported **Test → Manually** action while configuring the scheduled flow.
5. Add **Compose — SnapshotDate**:

```
formatDateTime(
  convertTimeZone(utcNow(), 'UTC', 'Eastern Standard Time'),
  'yyyy-MM-dd'
)
```

6. Add **Compose — RunId**:

```
workflow()['run']['name']
```

Initialize variables at the top level, before conditions/loops. Each slash-separated name below represents a separate variable/action:

Name	Type	Initial value
AxonBaseUrl	String	Private HTTPS agency origin, without trailing slash.
AgencyId / ClientId	String	Corresponding private identifiers.
CreationScopeFromDate	String	`2024-03-11`
CreationToDate	String	SnapshotDate output.
CreationFromDate	String	Empty; set in Step 8.
UpdatedFromUtc / UpdatedToUtc	String	Empty; set in Steps 8 and 11.
TokenExpiresUtc	String	Empty; populate from the validated token response.
OverlapSeconds	Integer	`86400`
PageSize	Integer	`50`
PageOffset	Integer	`0`
ReportPages / UpdatedReportPages	Array	Empty array `[]`.
FinishedCreated / FinishedUpdated	Boolean	`false`
LatestExists	Boolean	`false`
LatestFileId / WorkingFileId	String	Empty.
RefreshMode / PreviousRunId	String	Empty.
ForceFullReconciliation	Boolean	`true` for the initial pilot; later false.
LegacyTokenPartnerId	Boolean	`false`, subject to the tenant authentication check.
RunFailed / PageAccepted / TokenRefreshedForPage	Boolean	`false`
FailureReason	String	Empty.
MaxRetries / RetryCount	Integer	`6` / `0`.

Bind ClientSecret, AccessToken, and token expiry using the protected procedure in Step 6. URI/body fragments assume ClientSecret and AccessToken variables; substitute secure action outputs when your credential method uses them directly. Initialize sensitive placeholders empty and populate them after successful credential/token retrieval. Add the protected token-acquisition actions after initialization and before the first report GET.

Use strings for ISO dates/timestamps, integers for indexes, booleans for flags, and arrays for pages. Quoted `"false"` and `"[]"` are strings, not the desired types. Rename actions before copying expressions that reference their names.

Checkpoint: the flow saves and private values have not been placed in repository files.

Step 8 — Create a working copy and choose refresh mode

1. Add **Compose — AcquisitionStartedUtc**:

```
formatDateTime(utcNow(), 'yyyy-MM-ddTHH:mm:ss')
```

2. Locate/create raw/SnapshotDate, then create raw/SnapshotDate/RunId. An existing date folder is normal; check folder metadata and create only on a confirmed missing-folder response.
3. Add **SharePoint → Get file metadata using path**, name it **Get_Latest_Metadata**, for latest/Uncategorized Report.xlsx.
4. On success set LatestExists=true and LatestFileId from the file Identifier/Id.
5. Put lookup in a Scope and add a catch Scope using **Configure run after → failed/timed out**. Only a confirmed **404/file not found** sets LatestExists=false; other errors terminate Failed. Configure the subsequent mode-selection action to run after the catch Scope is **Succeeded or Skipped**, so a successful lookup also continues. A permissions failure must not select first-run behavior.
6. Select backfill if latest is absent, ForceFullReconciliation=true, or Step 18's full-reconciliation rule is due. Otherwise select incremental.

Backfill branch

1. Set RefreshMode=backfill, PreviousRunId to empty, and CreationFromDate=CreationScopeFromDate.
2. Set UpdatedFromUtc to AcquisitionStartedUtc.
3. Get template file content and create raw/SnapshotDate/RunId/Uncategorized Report.xlsx from it.
4. Save that new file Identifier/Id in WorkingFileId.

Backfill covers March 11, 2024 through SnapshotDate. Measure volume first: the MVP processes one complete Created bundle per run and does not implement staged/chunked backfill. If it exceeds connector/script capacity, add tested partitioning or migrate state to a SharePoint list/Dataverse before publication. Never truncate pages.

Incremental branch

1. Set RefreshMode=incremental.
2. Copy the published latest workbook into the new run folder, saving its new Identifier/Id as WorkingFileId.
3. Add **Excel Online → Run script**, name it **Read_Refresh_State**, use WorkingFileId, and select ReadRefreshState.
4. Verify creation_from_date is `2024-03-11` and sample_data is false for production. Start the first real backfill from the blank template, not a processed synthetic workbook.
5. Set PreviousRunId to:

```
body('Read_Refresh_State')?['result']?['previous_run_id']
```

6. Set CreationFromDate using `incremental_created_from_date` in the expression reference: max(scope start, previous Creation To Date minus one day).
7. Set UpdatedFromUtc using `incremental_updated_from_utc`: previous Updated Through UTC minus OverlapSeconds.

Read metadata from the same copy you process. Copying a template for an incremental run discards the ledger.

Checkpoint: WorkingFileId points to a new run workbook; refresh mode/windows are fixed. Latest has not been edited.

Step 9 — Add controlled HTTP handling for report pages

Use this structure for both Created and Updated page loops; give the Updated actions distinct names.

1. At each page start reset RetryCount=0, PageAccepted=false, and TokenRefreshedForPage=false.
2. Create an inner **Do until** retry loop with exit expression:

```
or(variables('PageAccepted'), variables('RunFailed'))
```

3. Put the GET in a request scope. Set its automatic Retry policy to **None** when implementing the manual policy below, so there is one retry controller.

4. Add a response-handling scope configured to run after the request scope **succeeds, fails, or times out**. HTTP 4xx/5xx normally marks an HTTP action failed.
5. Inspect the named HTTP action's actual statusCode; for Created use `outputs('Get_Created_Page')?['statusCode']`, and for Updated substitute Get_Updated_Page. A missing response/status or unrecoverable timeout fails the run.
6. Apply this policy:

Response	Handling
200	Validate the correct report schema, append the page once, and set PageAccepted=true.
401 on report GET	Refresh the token once, set TokenRefreshedForPage=true, and retry the same page. A second 401 fails.
401 on token POST	Fail authentication.
403	Fail and check Reports.read/agency access.
400 / 404	Fail and diagnose host, route, window, or bindings.
429	Delay for nonnegative integer Retry-After seconds; if absent/invalid, delay at least **300 seconds**. Retry the same page.
500–599	Retry with delays 1, 2, 4, 8, 16, then 32 seconds for six retries. Cap additional backoff at 60 seconds if the retry count increases.
Retry limit reached	Set RunFailed=true and FailureReason, then exit without accepting the page.

For a retry, check the six-retry bound, delay as required, and increment RetryCount. Keep PageOffset and the date window unchanged. Refresh a token before its tracked expiry when necessary.

Set fatal flags inside loops, then use **Terminate → Failed outside the loops**. Do not rely on Terminate inside a loop. Configure count/time limits for every Do until based on tenant limits and measured volume. A loop ending at its limit does not complete a report.

If parsing, projection, or append fails, do not mark the page accepted. The final gate checks successful loop status and the terminal-page flag, including failed/timed-out outcomes.

Checkpoint: retries cannot append a page twice or skip it, and all failure paths retain latest. Validate error branches with controlled test responses before relying on production failures to exercise them.

Step 10 — Collect EvidenceCreated pages

1. Reset PageOffset=0, ReportPages=`[]`, and FinishedCreated=false.
2. Add an outer **Do until**, name it **Fetch_Created**, and use:

```
or(variables('FinishedCreated'), variables('RunFailed'))
```

3. Inside it, use Step 9's retry/request structure and name the HTTP action **Get_Created_Page**. Set the GET URI using `report_uri` from the expression reference. Its route is:

```
GET {AxonBaseUrl}/api/v1/agencies/{AgencyId}/reports/data
    ?reportType=EvidenceCreated
    &fromDate={CreationFromDate}
    &toDate={CreationToDate}
    &pageSize=50
    &pageOffset={PageOffset}
```

4. Add Authorization using `authorization_header`: Bearer followed by the current token. Enable Secure Inputs/Outputs.
5. On HTTP 200 add **Parse JSON**, name it **Parse_Report_Page**, set Content to `body('Get_Created_Page')`, and paste automation/axon-report-page.schema.json into Schema.

6. Add **Data Operation → Select**, name it **Select_Report_Fields**, and set From to:

```
body('Parse_Report_Page')?['data']
```

7. Map these keys to the listed expressions:

Output key	Expression
evidenceId	`item()?['evidenceId']`
status	`item()?['status']`
dateUploaded	`item()?['dateUploaded']`
ownerGroups	`item()?['ownerGroups']`
categories	`item()?['categories']`
idExternal	`item()?['idExternal']`
evidenceType	`item()?['evidenceType']`
ownerBadgeId	`item()?['ownerBadgeId']`
ownerFirstName	`item()?['ownerFirstName']`
ownerLastName	`item()?['ownerLastName']`
ownerRole	`item()?['ownerRole']`
serialNumber	`item()?['serialNumber']`
dateRecordStart	`item()?['dateRecordStart']`

Validate before projection so an absent categories field cannot become a false uncategorized value. Keep arrays as arrays. Select reduces metadata without creating an action per evidence record.

8. Add **Append to array variable → ReportPages**. Enter this Value Expression:

```
json(concat(
  '{"page_offset":', string(variables('PageOffset')),
  ',"data":', string(body('Select_Report_Fields')),
  '}'
))
```

9. If `length(body('Parse_Report_Page')?['data'])` is 0, set FinishedCreated=true. Otherwise use **Increment variable → PageOffset** by **1**.
10. Set PageAccepted=true only after append and flag/index handling succeed. Continue after a short nonempty page.
11. Outside Fetch_Created, require its status=Succeeded, FinishedCreated=true, and RunFailed=false. Configure this gate to inspect failed/timed-out loop results too; otherwise terminate Failed before processing/publication.

Include the successful terminal `data: []` page. PageOffset is 0, 1, 2 rather than 0, 50, 100. CreationFromDate/CreationToDate filter recording creation, not upload dates.

Checkpoint: ReportPages contains every validated page plus the empty terminal page. A first successful empty page is a legitimate zero-result report.

Step 11 — Collect EvidenceUpdated pages

1. After Created completes, set UpdatedToUtc once:

```
formatDateTime(utcNow(), 'yyyy-MM-ddTHH:mm:ss')
```

2. Reset PageOffset=0, UpdatedReportPages=[], and FinishedUpdated=false.
3. Add an outer loop named **Fetch_Updated**, using the Created structure with an exit expression for FinishedUpdated or RunFailed.
4. Name its HTTP action **Get_Updated_Page** and use `updated_report_uri` from the expression reference:

```
GET {AxonBaseUrl}/api/v1/agencies/{AgencyId}/reports/data
    ?reportType=EvidenceUpdated
    &updatedFromDateTime={UpdatedFromUtc}
    &updatedToDateTime={UpdatedToUtc}
    &pageSize=50
    &pageOffset={PageOffset}
```

5. Validate using automation/axon-updated-report-page.schema.json. Name the Parse JSON action **Parse_Updated_Page**, set its Content to `body('Get_Updated_Page')`, then name the Select action **Select_Updated_Fields** and set its From to `body('Parse_Updated_Page')?['data']`.
6. Use the same Select keys. Preserve Updated's category objects; Created categories are strings. The script normalizes them.
7. Append to UpdatedReportPages using the Step 10 expression with `Select_Updated_Fields` substituted.
8. Stop only on a successful empty data array and retain that terminal page. Otherwise increment PageOffset by 1.
9. Require successful loop status, FinishedUpdated=true, and RunFailed=false outside the loop.

These UTC parameters use `yyyy-MM-ddTHH:mm:ss` without trailing Z. Freeze UpdatedToUtc during pagination. Inclusion can depend on category-specific modification timestamps: the greatest top-level dateModified is not a safe watermark.

The guide's Updated parameter table has inconsistent wording; its operation description/metadata example defines changed-metadata inclusion. Verify UTC-window/category behavior in the tenant pilot.

Checkpoint: both reports have complete successful page arrays. UpdatedToUtc is the processed request-window boundary, not an independently provided extraction timestamp or point-in-time snapshot guarantee.

Step 12 — Compose and archive the refresh envelope

Add these Compose actions outside the loops, in order. Paste into **Expression** so outputs are typed JSON objects. This avoids relying on editable code view or a pasted JSON text box automatically becoming an object.

Compose_Completed_Bundle:

```
json(concat(
  '{"schema_version":1,"report_type":"EvidenceCreated",',
  '"from_date":"', variables('CreationFromDate'),
  '","to_date":"', variables('CreationToDate'),
  '","page_size":', string(variables('PageSize')),
  ',"sample_data":false,"pages":', string(variables('ReportPages')),
  '}'
))
```

Compose_Completed_Updated_Bundle:

```
json(concat(
  '{"schema_version":1,"report_type":"EvidenceUpdated",',
  '"updated_from_datetime":"', variables('UpdatedFromUtc'),
  '","updated_to_datetime":"', variables('UpdatedToUtc'),
  '","page_size":', string(variables('PageSize')),
  ',"sample_data":false,"pages":',
  string(variables('UpdatedReportPages')),
  '}'
))
```

Compose_Completed_Refresh_Bundle:

```
json(concat(
  '{"schema_version":1,"mode":"', variables('RefreshMode'),
  '","previous_run_id":"', variables('PreviousRunId'),
  '","creation_from_date":"', variables('CreationScopeFromDate'),
  '","creation_to_date":"', variables('CreationToDate'),
  '","sample_data":false,"created":',
  string(outputs('Compose_Completed_Bundle')),
  ',"updated":', string(outputs('Compose_Completed_Updated_Bundle')),
  '}'
))
```

Create `evidence-api.json` in the current run folder with File Content:

```
string(outputs('Compose_Completed_Refresh_Bundle'))
```

automation/axon-refresh-bundle.schema.json describes the envelope, and the synthetic backfill file shows its shape. Protect source-containing actions for the authorized reporting audience.

Checkpoint: archived JSON has actual objects/arrays, numeric schema/page values, boolean sample_data=false, and both complete bundles. It contains no token or client secret.

Step 13 — Process and check the working workbook

1. Add **Excel Online (Business) → Run script**, rename it **Process_Evidence**, use WorkingFileId, and select ProcessEvidence.
2. Enter:

Parameter	Production value
csvText	`string(outputs('Compose_Completed_Refresh_Bundle'))`
snapshotDate	`outputs('SnapshotDate')`
runId	`outputs('RunId')`
sourceDataDate	Empty string (`''` in Expression), unless an authoritative extraction date is supplied.
sampleData	Boolean `false`.
sourceFormat	`axon-refresh-bundle`

3. Use supported retries for transient Excel file-availability/lock errors. This action operates on the working copy.
4. Add a Condition with:

```
and(
  equals(body('Process_Evidence')?['result']?['run_id'],
    outputs('RunId')),
  equals(body('Process_Evidence')?['result']?['snapshot_date'],
    outputs('SnapshotDate')),
  equals(body('Process_Evidence')?['result']?['sample_data'], false),
  equals(body('Process_Evidence')?['result']?['rejected_rows'], 0)
)
```

5. On failure terminate Failed and inspect the working workbook's Data Quality sheet. This guide's strict production default is zero rejected rows. A different publication policy needs a documented reporting-owner decision.

The script retains all statuses/categories in EvidenceState, applies Created then Updated by internal evidenceId, and recomputes ages across the ledger. Category assignment/deletion removes items from the backlog; category removal can restore them. Invalid IDs/recording-start dates or conflicting duplicates stop processing.

Checkpoint: run identity/date match, Sample Data=false, and the expected counts reconcile. Refresh-envelope runs have an eighth Evidence State sheet. Processing success does not yet publish latest.

Step 14 — Publish and verify the first live backfill

1. Get file content for WorkingFileId after Step 13's success gate.
2. If LatestExists=true, use **SharePoint → Update file**, selecting LatestFileId and the completed workbook bytes.
3. If LatestExists=false from the confirmed missing-file branch, **Create file** in latest named `Uncategorized Report.xlsx` with those bytes.
4. Confirm successful publication and compare latest's Run ID/Snapshot Date with the working workbook and flow result.
5. Compare API population and component/AOR totals against a representative manual export. Check recording-date boundaries and ownerGroups ordering: the legacy logic uses the first group, whose order is not guaranteed by the vendor guide.
6. Inspect upload ages 14, 15, 29, and 30 and confirm their bands.

Preserve latest's file identity on later runs by updating its contents. Each run keeps raw JSON and a working workbook in its archive. Publishing latest updates state, summaries, and the UTC watermark together; an archived processed workbook alone does not advance production state.

Checkpoint: a complete real backfill from March 11, 2024 is published to latest, with an authoritative run identity and successful quality gate.

Step 15 — Connect the Power BI project to SharePoint

1. Open the project or saved PBIX in Desktop.
2. Select **Transform data → Manage parameters** and set:

Parameter	Value
SharePointSiteUrl	HTTPS site home-page URL from Step 3.
ReportFolderPath	Actual library URL path plus folder, for example `Shared Documents/BWC Reporting`.
UseSampleData	Boolean `false`, after latest exists.

3. Sign in using the supported organizational SharePoint/OAuth method and the tenant's organizational privacy settings.
4. Select **Close & Apply**, then **Refresh**.
5. Confirm the source is the single latest workbook's Uncategorized, Snapshot, and DataQuality tables. EvidenceState and archived detail snapshots are not current-report facts.
6. Compare Run ID, Snapshot Date, Updated Through UTC, and counts against latest.
7. Test Overview filters, synchronized Remediation slicers, oldest-first sorting, and empty selection.

UseSampleData selects a source; the workbook's Sample Data flag controls its visible mode label. A synthetic SharePoint workbook still displays SYNTHETIC DEMO DATA. Keep badge/evidence/device identifiers as text.

Checkpoint: the report reads the real published workbook and displays that run's metadata.

Step 16 — Publish and configure Power BI Service

1. Save and **Publish** from Desktop to the authorized workspace.
2. In Power BI Service locate the report and its **semantic model**. Some connector/UI labels still say "dataset."

3. Open the model's **Settings → Data source credentials / cloud connections** and configure the SharePoint organizational OAuth connection. Labels vary by tenant/version.
4. Verify the source is the SharePoint HTTPS site. This design uses cloud SharePoint credentials and does not require a desktop gateway or local synchronized file.
5. Request a refresh, inspect **Refresh history** for completion, then reopen the report and compare run identity/date to latest.
6. Assign the intended audience. SharePoint permissions do not automatically become Power BI row-level security; the report includes owner-level details and rejected records. Hiding a page does not secure its rows.

Checkpoint: a completed Service refresh reads latest. Publishing, sharing, and publishing a Power BI app are separate operations.

Step 17 — Request refresh after successful publication

1. In the production flow, add **Power BI → Refresh a dataset**, or the tenant-supported equivalent, after latest's successful update/create branch.
2. Select the workspace and actual semantic model/dataset from Step 16. Use the model identifier, not the report identifier.
3. Connect using an identity authorized to request refresh.
4. Keep refresh on the publication-success path; failure branches must not request it.
5. Record the flow run identity and refresh-request result. Refresh is asynchronous: inspect model Refresh history for completion and verify the report's metadata.

This template directly imports the SharePoint workbook. If a dataflow is added later, complete its refresh before requesting the downstream model refresh.

Checkpoint: after a flow run, model refresh completes and the report shows that same published run. Request acceptance alone does not prove ingestion of the new bytes.

Step 18 — Enable recurrence and periodic full reconciliation

1. After real backfill succeeds, set ForceFullReconciliation=false for ordinary incremental runs.
2. Verify an incremental run copies latest, reads its state, and collects both overlapping windows.
3. Test unchanged-record aging, category assignment/removal, deletion, and overlap replay.
4. Retain concurrency=1 and enable the agreed daily/weekly schedule.
5. Add a full-backfill rule to the same mode selection in Step 8:

Recurrence	Starting rule
Daily	Full reconciliation each Sunday.
Weekly	Full reconciliation on the scheduled weekday within days 1–7 of each month.

Daily rule, combined with the missing-latest branch:

```
or(
  variables('ForceFullReconciliation'),
  equals(dayOfWeek(
    convertTimeZone(utcNow(), 'UTC', 'Eastern Standard Time')
  ), 0)
)
```

Weekly flow's monthly rule:

```
or(
  variables('ForceFullReconciliation'),
  lessOrEquals(int(formatDateTime(
    convertTimeZone(utcNow(), 'UTC', 'Eastern Standard Time'),
    'dd'
  )), 7)
)
```

On-demand runs can force full reconciliation. Adjust cadence/overlap based on volume and observed indexing delays. The one-day overlap is an MVP default, not a completeness guarantee. Full reconciliation repairs missed changes or disappeared evidence; the Updated example shows Deleted status but does not guarantee every permanent deletion remains retrievable.

Checkpoint: with the authoring desktop off, a scheduled run publishes a new workbook and a completed Service refresh shows matching metadata. Weekly runs use the last successful watermark to cover intervening days.

Step 19 — Monitor and recover

Event	Response
API/page acquisition failure	Retain latest; repair the cause and retry from its last successful state. Do not manually advance a watermark.
Processing/quality failure	Inspect archived raw JSON and Data Quality; resolve invalid fields or duplicate conflicts before publication.
SharePoint publication failure	Retain prior latest and retry after fixing access/lock errors. The archive alone is not the production base.
Latest publishes; Power BI refresh fails	New state is already published. Repair credentials/refresh and refresh again; a report-refresh failure alone does not require rolling back the workbook.
Report looks stale	Compare report versus latest Run ID/Snapshot Date; check completed Refresh history.
Restore a prior completed run	Disable recurrence, restore known-complete archived workbook bytes into the stable latest file, refresh the model, verify the restored run, and resume from that watermark.
Change approved creation scope	Perform full backfill. Incremental processing rejects a changed scope.

Monitor failed Power Automate runs and model Refresh history. Configure notifications only for agreed recipients/channels. Retain raw runs and SharePoint version history according to the organization’s retention rules.

Troubleshooting

Symptom	Check
version.json missing	Update clone/corrected ZIP; preserve adjacent folders and verify Report/definition/version.json.
Desktop still cannot open/render	Capture the exact error and Desktop version; native compatibility remains to be verified.
Automate tab/Run script unavailable	Check tenant enablement and licensing for Office Scripts/Excel Online.
Script not listed	Check script ownership/sharing and the connector identity.
Wrong workbook processed	Use WorkingFileId from Create file, not template/latest or numeric ItemId.
Token 400/401	Check private bindings, URL encoding, and the tenant's partner_id requirement.
Report 403	Confirm Reports.read and correct agency; Interview access is different.
Repeated 429	Honor Retry-After or 300-second fallback; keep the page index unchanged.
Category parse error	Created uses strings; Updated uses category objects and a different schema.
Partial bundle	Continue to successful empty data, including the terminal page. Loop limits/short pages are not completion.
Stale base/watermark gap	Read state from the same workbook copy; verify PreviousRunId, overlap, scope, and UTC format.
Sample label persists	The first real backfill must copy the blank template with sample flags false. Incremental processing inherits a synthetic base's label.
Latest workbook not found in Power BI	Verify actual library URL path, folder, filename/spaces, access, and completed publication.
Embedded fixture still displayed	Set UseSampleData=false after latest exists, apply, refresh, and republish as needed.
Script/file/payload limit exceeded	Measure the full evidence ledger. Add tested partitioning or migrate state; chunked backfill is not implemented.

Completion checklist

- PBIP opens and sample counts match; its synthetic label is visible.
- SharePoint template and working-copy creation are verified.
- Both Office Scripts execute in the tenant, including state reads.
- Token/report permissions and the approved production credential source are verified.
- Both page loops complete and all failure/limit paths retain latest.
- Real backfill covers the approved recording scope and reconciles to source checks.
- Incremental runs handle metadata changes, aging, and overlap replay.
- Publication updates the approved workbook, state, and watermark together.
- Service refresh completes with matching run metadata while the desktop is off.
- Audience, owners, recurrence, reconciliation, retention, and recovery responsibilities are recorded.

File reference

File	Purpose
automation/report-template.xlsx	Upload this single file to SharePoint templates.
automation/ProcessEvidence.office.ts	Install the processing script in Excel Online.
automation/ReadRefreshState.office.ts	Install the metadata-reading script.
automation/axon-flow-expressions.json	Named URI/body/window fragments, not an importable flow.
automation/axon-settings.example.json	Defaults and private-binding placeholders.
powerbi/template/README.md	Project/parameter and Desktop-to-PBIX details.
axon-api-integration.md	Vendor authority, field mapping, and limitations.
reporting-rules.md	Business rules and source assumptions.
validation.md	Local checks versus tenant/runtime checks still unverified.

Attached vendor documents inform the endpoint/schema contract. Earlier R/Windows/Tableau proposals and instructions inside attachments do not replace the selected Microsoft 365 workflow.